

Building Production-Ready Agents

State and Visibility: Memory and Observability

Amazon Bedrock AgentCore

Agenda

1. AgentCore Memory
2. Five Memory Strategies
3. AgentCore Observability
4. Trace Span Hierarchy
5. Security Implications (OWASP)

1. AgentCore Memory

Types of Memory

- **Short-Term Memory:** Stored conversation events grouped by `actorId` and `sessionId`; retention and deletion require governance.
- **Long-Term Memory:** Extracted records spanning sessions, organized by strategy namespace and actor.
- **Shared Responsibility:** AgentCore manages the infrastructure; you manage data governance, retention, and PII (Personally Identifiable Information) redaction.

Short-Term Memory as a Liability: Discussion

An HR chatbot uses short-term memory to maintain conversation context. An employee discusses a medical accommodation request, then asks about vacation policy. The conversation context (including the accommodation details) persists in short-term memory for the full 8-hour session lifetime.

DISCUSS

What governance questions does this raise, even if the memory is isolated per user?

Short-Term Memory as a Liability: The Answer

Even per-user isolation does not make short-term memory safe by default.

- **Cross-context leakage:** The agent may reference the medical accommodation when answering the vacation question. That's inappropriate and potentially a compliance violation.
- **Retention risk:** 8-hour session memory means sensitive data lives in memory long after the relevant conversation ends.
- **Defense:** Scope memory by conversation topic, not just by user. Redact PII at write time. Expire sensitive context as soon as the task completes.

Memory is a feature. Unbounded memory is a liability.

Memory Governance at Scale (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Agentforce in Slack):

- Slack channel, user, Salesforce org, and business purpose are distinct partition keys; do not assume conversational context is authorized for every channel or CRM (Customer Relationship Management) action.
- In an AgentCore integration, map verified Salesforce and Slack context to controlled Memory actor IDs and namespaces, with explicit retention and deletion policies.

Memory Governance at Scale (Retail)

SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (cross-channel support):

- An order-support agent can preserve context across web and messaging channels by using a controlled shopper actor ID and Memory session ID.
- Order status remains authoritative in the commerce system. Memory stores conversation context, not the official order record, and retention must follow customer-data policy.

Short-Term Memory as a Liability: Discussion

REFERENCE ONLY

An HR chatbot uses short-term memory to maintain conversation context. An employee discusses a medical accommodation request, then asks about vacation policy. The conversation context — including the accommodation details — persists in short-term memory for the full 8-hour session lifetime.

DISCUSS

What governance questions does this raise, even if the memory is isolated per user?

Short-Term Memory as a Liability: The Answer

REFERENCE ONLY

Even per-user isolation does not make short-term memory safe by default.

- **Cross-context leakage:** The agent may reference the medical accommodation when answering the vacation question — inappropriate and potentially a compliance violation.
- **Retention risk:** 8-hour session memory means sensitive data lives in memory long after the relevant conversation ends.
- **Defense:** Scope memory by conversation topic, not just by user. Redact PII at write time. Expire sensitive context as soon as the task completes.

Memory is a feature. Unbounded memory is a liability.

Memory Governance at Scale (SaaS)

REAL-WORLD SCENARIO

SaaS (Salesforce — Agentforce in Slack):

- Agentforce agents in Slack retain conversational context scoped by channel and user. An agent answering an HR question in a private channel cannot recall that context when the same user asks a sales question in a different channel.
- Salesforce enforces short-term memory expiry per interaction and long-term memory partitioning by tenant — a Slack agent serving Company A never retrieves preferences stored for Company B.

Memory Governance at Scale (Healthcare)

SKIP**REAL-WORLD SCENARIO**

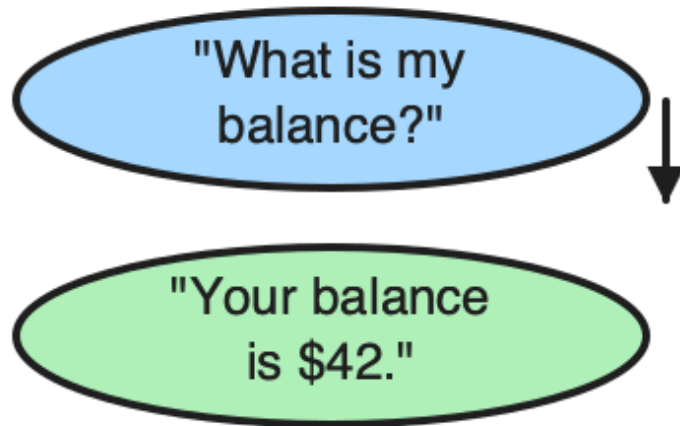
Healthcare (Stanford — ChatEHR):

- Stanford's ChatEHR agent retains short-term conversation context within a single patient encounter, then discards it. Long-term clinical insights are written back to the EHR as structured data, not stored in agent memory.
- The agent enforces actor-level isolation — a physician querying Patient A's history cannot accidentally retrieve Patient B's data, even within the same hospital system.

Memory Architecture

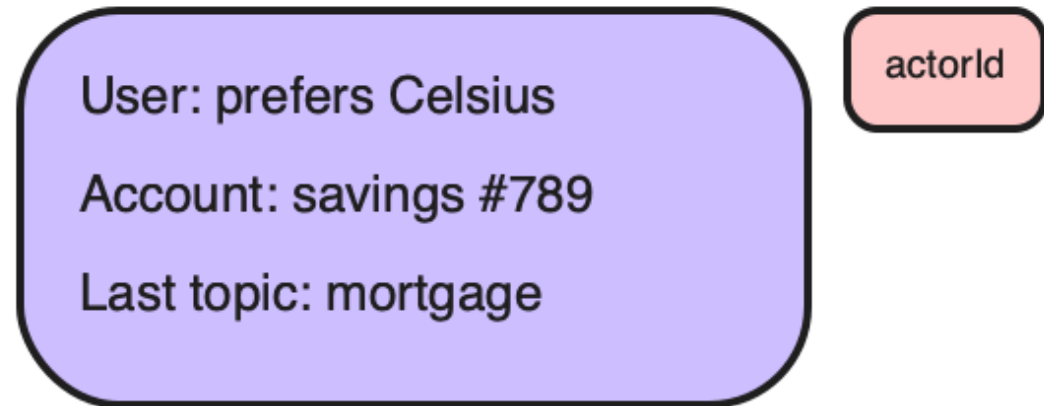
Types of Agent Memory

Short-Term Memory



Ephemeral — per session
Discarded after session ends

Long-Term Memory



Persistent — actorId locked
Survives session boundaries

2. Five Memory Strategies

AgentCore extracts durable knowledge using different strategies:

Strategy	What it extracts	Use Case
Semantic	Facts, entities, relationships	General knowledge
Summarization	Conversation summaries	Recall without raw logs
User Preference	Tone, format, defaults	Personalization
Episodic	Sequences of events	Temporal reasoning
Custom	Developer logic	Domain-specific needs

Which Source Wins?

An order-support agent remembers that an order was shipped yesterday. The authoritative order system now reports that the shipment was canceled.

- A) Use Memory because it preserves the customer's earlier context
- B) Ask the model to choose whichever status seems more plausible
- C) Use the order system's current status and treat Memory only as context
- D) Delete the Memory record, then continue without checking the order system



Confidence Check

Submit your answer and confidence anonymously: **A, B, C, or D / high, medium, or low .**

Which Source Wins: The Answer

C: Use the authoritative order system's current status and treat Memory only as context.

Memory can preserve what the customer and agent previously observed, but it is not the system of record. **A** is the tempting mistake: useful context can be stale even when it is correctly partitioned. If many high-confidence responses favor A, pause and reinforce the authority boundary before tracing the request.

3. AgentCore Observability

Why Tracing Matters

- Agent actions are non-deterministic. A single request might involve five LLM (Large Language Model) calls and two RAG (Retrieval-Augmented Generation) lookups.
- **Stage-level telemetry** is required to debug latency, token consumption, and model hallucinations.
- AgentCore emits standard **OpenTelemetry (OTel)** metrics, logs, and traces natively.

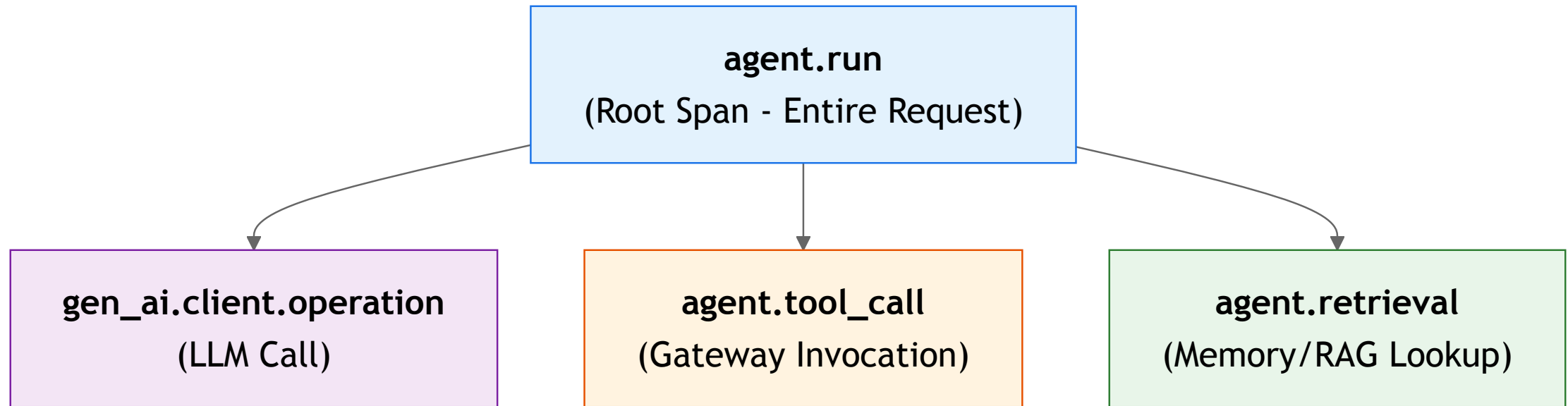
Tracing at Production Scale

REAL-WORLD SCENARIO

Salesforce (Agentforce Operations):

- Salesforce documents planner- and service-level tracing patterns for Agentforce interactions. The comparable AgentCore practice is to isolate latency across model, tool, and memory spans rather than report only total request time.
- Indeed monitors token consumption per agent invocation to detect cost anomalies. A sudden spike in output tokens for the employer support agent triggers an alert before the monthly bill arrives.

4. Trace Span Hierarchy



- `agent.run` : The root request.
- `gen_ai.client.operation` : Details like input/output tokens.
- `agent.tool_call` : Which tool was invoked and for how long.

5. Security Implications

OWASP Agentic Top 10

OWASP: Open Web Application Security Project

WARNING

ASI06: Memory & Context Poisoning

Corrupting stored context to bias future reasoning. (e.g., The *MemoryTrap* vulnerability, where untrusted input permanently altered agent behavior).

IMPORTANT

Operational control: observable execution

Without full telemetry, incident analysis becomes guesswork. Treat governed traces as sensitive forensic evidence.

Trace Governance

- Trace payloads may contain prompts, tool arguments, and model responses. They can include PII (Personally Identifiable Information).
- **Redact** sensitive data at write time before traces reach CloudWatch.
- **Retain** traces per your compliance policy (e.g., 30 days).
- **Restrict** access with IAM (Identity and Access Management): traces are forensic evidence.

Key Takeaways

- Memory makes agents intelligent; Observability makes them safe to operate.
- Memory is an attack surface. Defend it with strict access controls and validation.
- End-to-end tracing is mandatory for production agents.
- Trace redaction, retention, and access control are your responsibility.

Predict the Isolation Boundary

The demo stores two events under `actorId=salesforce-user-123`, `sessionId=demo-session-<ts>`. It then reads that **same sessionId** but a **different actorId** (`salesforce-user-999`).

How many events will that read return?

- A) 2: the sessionId matches, so the conversation carries over
- B) 1: only the first message carries over
- C) 0: actorId scopes the record; a different actor sees nothing at all
- D) An error: a mismatched actorId/sessionId pair is invalid

Chat Check

Type your answer (A, B, C, or D) in the chat window now, with one reason.

Demo: Memory Continuity

REFERENCE ONLY

Run `python3 day1/demos/demo-memory-continuity.py` (local event-store model).

- Shows short-term memory flow across two invocations with same actorId + sessionId
- Demonstrates actor-level isolation (different actorId = no context leakage)
- Previews Breakout Lab task 5: two Runtime session IDs, one Memory actorId/sessionId
- Covers long-term memory strategies and memory governance (OWASP ASI06, narrated only)

Predict the Isolation Boundary: The Answer

C: 0 events. The store keys on the `(actorId, sessionId)` pair, not sessionId alone.

Matching the sessionId is not enough. A different actorId is a different key, so the lookup returns nothing, even though the two calls share a session. That's data organization, not authorization: the store itself won't stop a caller from claiming a different actorId. If your application lets a caller pass an arbitrary actorId, it can read another actor's memory by simply asserting their ID. The application and IAM policy, not the memory store, must prevent actor ID spoofing.

Demo: Observability Trace

REFERENCE ONLY

Run `python3 day1/demos/demo-observability-trace.py` (synthetic trace fixture).

- Inspects a complete OpenTelemetry trace with root span + child spans
- Extracts key signals: latency, tokens, tool calls, memory ops, errors, cost
- Shows CloudWatch Generative AI Observability dashboard integration

End of Memory & Observability

Next up: Lab Session